Contents

1 Chapter 1-2 Notes **2**

2 Chapter 3 **3**
2.1 Variables and Mutability 3
2.2 Data Types 4

1 Chapter 1-2 Notes

Skipped this portion, basic syntax...

2 Chapter 3

Definition – Keywords

Reserved set of words that a programming language can use.

2.1 Variables and Mutability

By default, variables are immutable. A key aspect of that makes writing safer and concurrent code easier.

Immutability is the state of a variable bounded to a particular value that was initially set. Where the Rust book states:

> If one part of our code operates on the assumption that a value will never change and another part of our code changes that value, it's possible that the first part of the code won't do what it was designed to do.

If we did want to make it mutable, we can do so by adding the keyword, `mut`. This also helps future readers of the code know that the variable will change throughout.

Constants, similar to immutable variables, are variables bounded to an initially set value. However, there are some key differences:

- Cannot use `mut` → they are ALWAYS immutable
- Uses `const` keyword instead of `let`
- Can be declared in ANY scope, including global.

These make code more readable and help readers know what values mean. In example:

```
const THREE_HOURS_IN_SECONDS: u32 = 60 * 60 * 3;
```

Shadowing is a term to describe when the first variable is 'shadowed' by the second. Where we can scope the same named variable and have it have different values.

For example:

```
fn main() {
    let x = 5;
    let x = x + 1;
        {
            let x = x * 2;
            println!("The value of x in the inner
                    scope is: {x}");
        }
    println!("The value of x is: {x}");
}
```

So in this example, x will be initiated at 5, then reset to 6 by using the keyword `let` again... Then in the inner brackets, it also shadows the first two again, therefore setting the value to 12.

The main difference between `mut` and shadowing is that we are effectively creating new variables via shadowing...
This is particularly useful when we want a variable to possibly hold two types throughout the code, i.e. a string and then figuring out the string length, all under the same variable.

2.2 Data Types

Data types are types that tell Rust what kind of data is being specified. Moreover, there are two data type subsets, scalar and compound.
Rust is a statically typed language!! This means that it must know the types of all variables at compile time. For example:

```
let guess: u32 = "42".parse().expect("Not a number!");
```

This would throw a compiling error without the 'u32' as it needs more information as to what the variable type is.

Scalar types are types that represent a singular value. Where Rust has four primary scalar types: integers, floats, Booleans, and chars.

Compound types are types that can group multiple values into one type. Particularly Rust has tuples and arrays.

Here is an example of a tuple:

```
fn main() {
    let tup: (i32, f64, u8) = (500, 6.4, 1);
}
```

Since tup is considered a single compound value, we can extract the values as such:

```
fn main() {
    let tup = (500, 6.4, 1);
    let (x, y, z) = tup;
    println!("The value of y is: {y}");
}
```

Alternatively, we can access the values, similar to an array:

```
fn main() {
    let x: (i32, f64, u8) = (500, 6.4, 1);
    let five_hundred = x.0;
    let six_point_four = x.1;
    let one = x.2;
}
```

I got lazy, but the rest of this chapter is the same as many other programming languages :D